Министерство образования и науки Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Санкт- Петербургский политехнический университет Петра Великого»
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
02.03.03 Математическое обеспечение и администрирование информационных систем

Отчет по выполнению курсовой работы по дисциплине Курсовое проектирование по объктно-ориентированному программированию

Создание телефонного справочника с использованием библиотеки Qt

Выполнил: студент группы 5130203/40001 Павлова С.С.
Принял: Преподаватель Йовановски Ненад

Санкт- Петербург, 2026

Содержание

ВВЕДЕНИЕ..... 4

1. ПОСТАНОВКА ЗАДАЧИ 5

2. РЕАЛИЗАЦИЯ 7

2.1. Задача 1. Консольная версия 7

2.1.1. Структура приложения 7

2.1.2. Хранение данных 7

2.1.3. Реализация валидации 7

2.1.4. Функциональные возможности 8

2.2. Задача 2. Реализация с помощью Qt 8

2.2.1. Структура приложения 8

2.2.2. Вспомогательные классы 8

2.2.3. Используемые сторонние классы Qt 9

2.2.4. Сигналы и слоты 11

2.2.5. Описание функций класса Contact 12

2.2.5.1. toDisplayList 12

2.2.5.2. phonesAsString 12

2.2.6. Описание функций класса ContactValidator 12

2.2.6.1. isValidName 13

2.2.6.2. normalizePhone 13

2.2.6.3. normalizeName 13

2.2.6.4. isValidEmail 13

2.2.6.5. isValidPhoneFormat 14

2.2.6.6. validatePhoneWithDetails 14

2.2.6.7. isValidBirthDate 14

2.2.7. Описание функций класса MainWindow 14

2.2.7.1. MainWindow(QWidget *parent) 14

2.2.7.2. ~MainWindow() 15

2.2.7.3. setupUI 15
2.2.7.4. onSearchFieldChanged 15
2.2.7.5. onSearchTextChanged 15
2.2.7.6. createActions 16
2.2.7.7. createMenus 16
2.2.7.8. updateTable 16
2.2.7.9. getCurrentRow 17
2.2.7.10. getPhoneType 17
2.2.7.11. editPhones 17
2.2.7.12. editContact 18
2.2.7.13. onAddContact 19
2.2.7.14. onEditContact 20
2.2.7.15. onDeleteContact 21
2.2.7.16. onSave 22
2.2.7.17. onLoad 23
2.2.7.18. saveToFile 23
2.2.7.19. loadFromFile 24
2.3. Задача 3. Добавление поддержки PostgreSQL 24
2.3.1. Архитектура хранилищ 24
2.3.2. Интеграция в основное приложение 25
2.3.3. Обработка ошибок подключения 26
2.3.4. Сборка проекта 26
3. ТЕСТИРОВАНИЕ ПРОГРАММЫ 27
ЗАКЛЮЧЕНИЕ 33

Введение

В данной работе для создания приложения с графическим интерфейсом используется библиотека Qt. Она предоставляет широкий набор инструментов, таких как Qt Widgets для построения интерфейсов, QTableView и QStandardItemModel для отображения табличных данных, а также QJsonDocument для сериализации информации в формат JSON. Эти компоненты позволяют реализовать функциональный и интуитивно понятный телефонный справочник.

Особое внимание уделяется проверке вводимых пользователем данных с помощью регулярных выражений (QRegularExpression), что обеспечивает корректность имён, адресов электронной почты и телефонных номеров. Механизм сигналов и слотов в Qt используется для обработки событий и взаимодействия между элементами интерфейса. Приложение поддерживает несколько телефонных номеров различных типов для каждого контакта, выполняет валидацию данных и предоставляет удобные средства для управления записями.

1. Постановка задачи

Цель работы: разработать приложение «Телефонный справочник» в трех вариантах: в консольной версии, с графическим интерфейсом на основе фреймворка Qt и с возможностью хранения в базе данных, обеспечивающее хранение, редактирование, поиск и валидацию контактных данных.

Основные задачи:

1. Разработка структуры данных для хранения контактов.
   1.1. Определить обязательные поля для каждого контакта: фамилия, имя, отчество (опционально), адрес (опционально), дата рождения, адрес электронной почты (опционально).
   1.2. Реализовать поддержку неограниченного количества телефонных номеров для каждого контакта с указанием их типа (рабочий, домашний, служебный, мобильный).
   1.3. Обеспечить хранение номера телефона в нормализованном формате (только цифры) с сохранением исходного форматирования для отображения.

2. Реализация модуля валидации и нормализации вводимых данных.
   2.1. Разработать алгоритмы проверки корректности всех полей контакта:
   - Имя, фамилия, отчество: должны начинаться с заглавной буквы, могут содержать буквы, дефисы и пробелы, дефис не может находиться в начале или конце, после дефиса должна следовать заглавная буква.
   - Телефонный номер: должен соответствовать международным форматам, содержать только допустимые символы (цифры, пробелы, скобки, дефисы, знак «+» в начале), для российских номеров (начинающихся на 7 или 8) - содержать ровно 11 цифр.
   - Адрес электронной почты: должен соответствовать стандартному формату `user@domain.tld`.
   - Дата рождения: должна быть корректной, существующей и предшествовать текущей дате.
   2.2. Реализовать функции нормализации: удаление лишних пробелов в текстовых полях и приведение телефонных номеров к каноническому виду (только цифры).

3. Создание графического пользовательского интерфейса (GUI).
   3.1. Разработать главное окно приложения с табличным представлением списка контактов, содержащим колонки: Фамилия, Имя, Отчество, Адрес, Дата рождения, Email, Телефоны.
   3.2. Настроить модель данных (QStandardItemModel) для управления данными таблицы, то есть задать заголовки, установить связь с внутренней структурой данных, дать возможность включение сортировки по клику на заголовок столбца (для даты - по хронологическому порядку).
   3.3. Реализовать панель меню и панель инструментов для доступа к основным операциям.

4. Реализация диалоговых окон для операций с контактами.

4.1. Создать универсальное диалоговое окно для добавления и редактирования контакта с полями ввода и валидацией.

4.2. Разработать специализированный диалог для управления списком телефонных номеров контакта (добавление, изменение типа, удаление).

4.3. Реализовать диалог подтверждения удаления контакта для предотвращения случайного удаления.

5. Разработка системы динамического поиска и фильтрации.

5.1. Реализовать интерфейс для поиска, включающий поле ввода текста и выпадающий список для выбора области поиска (все поля или конкретный столбец).

5.2. Обеспечить динамическую фильтрацию таблицы в реальном времени при вводе текста.

5.3. Реализовать алгоритм поиска, поддерживающий:

* Поиск по всем полям одновременно или по выбранному столбцу.
* Частичное совпадение текста (регистронезависимое).
* Поиск по телефонным номерам только по цифрам, игнорируя форматирование (скобки, пробелы, дефисы).

6. Реализация системы сериализации и работы с файлами.

6.1. Обеспечить сохранение и загрузку всего списка контактов в формат JSON с использованием классов Qt (QJsonDocument, QJsonArray, QJsonObject).

6.2. Сохранять все поля контакта, включая вложенный список телефонов.

6.3. Использовать форматирование JSON с отступами (pretty- print) для удобочитаемости файла при его открытии в текстовом редакторе.

6.4. Реализовать интерфейс выбора файла для операций сохранения/загрузки с обработкой возможных ошибок ввода- вывода и информированием пользователя о результате операции.

7. Обеспечение корректности данных и логики приложения.

7.1. Реализовать обязательные проверки: наличие хотя бы одного телефонного номера, обязательность заполнения полей «Фамилия» и «Имя».

7.2. Связать элементы интерфейса с логикой приложения с использованием механизма сигналов и слотов Qt для обработки действий пользователя.

Ожидаемый результат

Программа должна предоставлять следующий функционал:

- Отображение списка контактов в табличной форме с семью колонками
- Добавление, редактирование и удаление контактов через диалоговые окна
- Сортировка по любому полю кликом на заголовок колонки
- Динамический поиск с фильтрацией по всем полям или конкретному полю
- Сохранение данных в JSON- файл и загрузка из него
- Комплексная валидация всех вводимых данных с информативными сообщениями об ошибках
- Поддержка неограниченного количества телефонных номеров различных типов

2. Реализация
2.1. Задача 1. Консольная версия

Перед разработкой графического интерфейса была реализована консольная версия приложения, которая послужила основой для отработки логики работы с данными, валидации и файлового ввода- вывода.

2.1.1. Структура приложения

Консольная версия реализована на стандартном C++ без использования фреймворка Qt. Архитектура включает следующие компоненты:

1) Класс Contact. Структура данных контакта (поля ФИО, адрес, дата рождения, email, список телефонов),
2) Класс PhoneInfo. Структура телефонного номера с указанием типа,
3) Класс ContactValidator. Класс со статическими методами для валидации всех полей,
4) Класс FileStorage. Класс для сохранения и загрузки данных в текстовый файл.

2.1.2. Хранение данных

В отличие от графической версии, где используется JSON, консольная версия применяет простой текстовый формат с разделителями. Формат строки: каждое поле отделяется символом |, телефоны разделяются символом ;, а тип и номер телефона - символом :. Такой формат прост в парсинге и не требует подключения дополнительных библиотек.

2.1.3. Реализация валидации

Валидация данных в консольной версии реализована с использованием стандартных регулярных выражений C++ (<regex>) и собственных функций для работы с UTF- 8 строками.

Особенности валидации:
ФИО - проверка на заглавную первую букву (с поддержкой кириллицы через ручной разбор UTF- 8), допустимые символы: буквы, пробелы, дефисы. Запрещены двойные дефисы и пробелы, а также дефис в начале или конце строки.

Телефон - поддерживаются форматы:
+78121234567 (11 цифр)
88121234567 (11 цифр)
+7(812)1234567
8(812)1234567
+7(812)123- 45- 67
8(812)123- 45- 67

Для российских номеров проверяется наличие ровно 11 цифр.

Email - проверка регулярным выражением на соответствие формату user@domain.tld.

Дата рождения - проверка на существование даты (с учётом разного числа дней в месяцах и високосных годов), а также на то, что дата предшествует текущей.

2.1.4. Функциональные возможности

Консольное приложение предоставляет следующий набор функций (доступных через текстовое меню):

Пункт	Описание
1	Показать все контакты (табличное отображение)
2	Добавить контакт с пошаговым вводом данных
3	Редактировать выбранный контакт
4	Удалить контакт с подтверждением
5	Поиск контактов (по всем полям или по конкретному)
6	Сортировка контактов (по фамилии, имени, дате рождения, email)
7	Сохранить справочник в файл
8	Загрузить справочник из файла
0	Выход с возможностью сохранения

2.2. Задача 2. Реализация с помощью QT

2.2.1 Структура приложения

Приложение "Телефонный справочник" реализовано с использованием объектно-ориентированного подхода на базе фреймворка Qt. Архитектура приложения включает следующие основные классы с иерархией:

QObject (базовый класс Qt) → QWidget → QMainWindow → MainWindow (главное окно приложения)

MainWindow является центральным классом приложения и наследуется от QMainWindow. Наследование выбрано по следующим причинам:

1. QMainWindow предоставляет готовую структуру главного окна с меню, панелями инструментов и центральным виджетом.
2. Автоматическая обработка событий и сигналов через механизм мета- объектов Qt.
3. Встроенная поддержка системы меню и панелей инструментов.
4. Возможность использования центрального виджета для размещения основного содержимого.

2.2.2 Вспомогательные классы

Contact (структура данных контакта) → PhoneInfo (структура данных телефона) → ContactValidator (валидатор данных)

Contact и PhoneInfo реализованы как структуры, так как не требуют наследования и представляют собой простые контейнеры данных. ContactValidator реализован как класс со статическими методами, что позволяет использовать его без создания экземпляров.

2.2.3. Используемые сторонние классы Qt

Для реализации приложения использованы следующие классы библиотеки Qt, сгруппированные по функциональному назначению:

Классы для создания интерфейса и управления окнами:

QMainWindow – базовый класс главного окна приложения, предоставляющий структуру с меню, панелью инструментов и центральным виджетом. Является родительским классом для MainWindow.

QWidget – базовый класс для всех элементов пользовательского интерфейса.

QDialog – базовый класс для диалоговых окон (окон редактирования и управления телефонами).

QMessageBox – диалоговое окно для вывода информационных, предупреждающих сообщений и запросов подтверждения.

QInputDialog – диалоговое окно для простого ввода данных пользователем (текст, выбор из списка).

Классы для работы с данными и их отображения:

QTableView – виджет для табличного представления данных, используется для отображения списка контактов.

QStandardItemModel – модель данных для таблицы, осуществляет хранение, управление и предоставление данных для QTableView.

QListWidget – виджет для отображения списка элементов, используется в диалоге управления телефонами.

Классы для компоновки элементов интерфейса (Layouts):

QVBoxLayout – менеджер компоновки, располагающий виджеты вертикально. Используется как основная компоновка окон.

QHBoxLayout – менеджер компоновки, располагающий виджеты горизонтально. Используется для размещения элементов поиска.

QFormLayout – менеджер компоновки, организующий виджеты в форме с метками (названия полей) и полями ввода. Используется в диалоге редактирования контакта.

Классы элементов управления (Widgets):

QLineEdit – однострочное поле для ввода текста (поле поиска, поля ввода данных).

QComboBox – выпадающий список для выбора значения (выбор поля поиска).

QPushButton – кнопка для инициации действий.

QDateEdit – поле для ввода даты с поддержкой выпадающего календаря. Используется для ввода даты рождения.

QCalendarWidget – виджет календаря, интегрированный в QDateEdit.

QGroupBox – контейнер с рамкой и заголовком для визуальной группировки элементов.

QHeaderView – заголовок таблицы, управляет отображением и поведением столбцов QTableView.

QDialogButtonBox – контейнер для стандартных кнопок диалога (OK, Cancel).

Классы для работы с файлами и сериализацией данных (JSON):

QFile – интерфейс для работы с файлами, используется для операций сохранения и загрузки.

QJsonDocument – класс для работы с JSON- документами, выполняет сериализацию и десериализацию данных.

QJsonObject – класс для представления JSON- объекта (один контакт или один телефон).

QJsonArray – класс для представления JSON- массива (список контактов или список телефонов).

QIODevice – базовый интерфейс для операций ввода- вывода, который наследует QFile.

Вспомогательные классы и шаблоны:

QString – класс для работы со строками в кодировке Unicode, используется для всех текстовых данных.

QDate – класс для работы с датами, используется для хранения даты рождения.

QList<T> – шаблонный класс- контейнер для хранения списков объектов (список контактов, список телефонов).

QAction – класс, представляющий действие, которое может быть привязано к пунктам меню, кнопкам панели инструментов или сочетаниям клавиш.

QMenuBar, QMenu – классы для создания и управления строкой меню и вложенными меню.

QToolBar – класс для создания панели инструментов.

QFileDialog – диалоговое окно для выбора файла для операций загрузки и сохранения.

QModelIndexList – список индексов модели, используется для обработки выбранных строк в таблице.

QAbstractItemView – базовый класс для видов элементов, предоставляет настройки поведения таблицы.

2.2.4. Сигналы и слоты

В приложении используется механизм сигналов и слотов Qt для реализации реактивного программирования. Этот механизм позволяет организовать слабосвязанное взаимодействие между объектами.

Основные соединения сигналов и слотов:

QLineEdit::textChanged() → MainWindow::onSearchTextChanged() - фильтрация контактов при вводе текста в поле поиска.

QComboBox::currentIndexChanged() → MainWindow::onSearchFieldChanged() - изменение поля поиска для фильтрации по конкретному столбцу.

QAction::triggered() → MainWindow::onAddContact() - добавление нового контакта.

QAction::triggered() → MainWindow::onEditContact() - редактирование выбранного контакта .

QAction::triggered() → MainWindow::onDeleteContact() - удаление выбранного контакта.

QAction::triggered() → MainWindow::onSave() - сохранение справочника в файл.

QAction::triggered() → MainWindow::onLoad() - загрузка справочника из файла.

QDialogButtonBox::accepted() → QDialog::accept() - подтверждение диалога (OK).

QDialogButtonBox::rejected() → QDialog::reject() - отмена диалога (Cancel).

2.2.5. Описание функций класса Contact

2.5.1. toDisplayList

Преобразует данные контакта в форматированный список строк для отображения в таблице. Используется в MainWindow::updateTable() для заполнения строк таблицы данными контактов.

2.5.2. phonesAsString

Формирует удобочитаемую текстовую строку, объединяющую все телефонные номера контакта для отображения в одном столбце таблицы. Для этого создаётся пустой список строк, для каждого объекта PhoneInfo в списке phones контакта вызывается его метод displayString() (который возвращает строку в формате "<тип>: <номер>"), полученная строка добавляется в список. Все строки из списка объединяются в одну результирующую строку с помощью разделителя "; ". Возвращается полученная строка.

2.2.6. Описание функций класса ContactValidator

Валидация данных в приложении построена на использовании регулярных выражений (Regular Expressions), предоставляемых классом QRegularExpression. Регулярное выражение - это шаблон, описывающий набор строк согласно определённым синтаксическим правилам. Валидация выполняется путём сопоставления входной строки с заданным шаблоном.

Основные элементы синтаксиса, использованные в работе:

\^ – якорь начала строки. Означает, что сопоставление должно начинаться с первого символа строки.

$ – якорь конца строки. Означает, что сопоставление должно продолжаться до конца строки.

\d – сокращение для класса символов [0- 9] (любая цифра).

\s – любой пробельный символ (пробел, табуляция).

* – квантификатор, означающий «0 или более раз».

+ – квантификатор, означающий «1 или более раз».

? – квантификатор, означающий «0 или 1 раз».

{n,m} – квантификатор, означающий «от n до m раз».

[...] – класс символов. Сопоставляется с любым одним символом, указанным внутри скобок. Например, [a- zA- Z] соответствует любой латинской букве.

\p{Lu}, \p{Ll} – свойства Unicode, сопоставляемые с буквами в верхнем (Lu) и нижнем (Ll) регистре соответственно, что позволяет работать с нелатинскими алфавитами (например, кириллицей).

() – группировка подвыражений.

\. – экранированная точка (символ '.').

2.2.6.1. isValidName

Проверяет корректность имени, фамилии или отчества согласно правилам русских имен. Функция валидирует строку имени по нескольким критериям:

1. Не должна быть пустой после удаления пробелов.
2. Должна начинаться с заглавной буквы.
3. Может содержать только буквы, пробелы и дефисы.
4. Не может начинаться или заканчиваться дефисом.
5. Не может содержать двойные дефисы или пробелы.
6. После дефиса должна идти заглавная буква.

Используемое регулярное выражение ^\p{Lu}[\p{Ll}\s\-]*$:

Строка должна начинаться с одной заглавной буквы любого алфавита, после которой может следовать ноль или более символов, где каждый следующий символ может быть: строчной буквой любого алфавита, ИЛИ пробельным символом, ИЛИ дефисом

2.2.6.2. normalizePhone

Приводит номер телефона к каноническому виду - только цифры без форматирования. Удаляет все нецифровые символы, включая знак "+", оставляя только цифры номера.

2.2.6.3. normalizeName

Удаляет лишние пробелы в имени, оставляя по одному пробелу между словами. Убирает пробелы по краям и заменяет множественные пробелы внутри строки на одинарные.

2.2.6.4. isValidEmail

Проверяет корректность формата email- адреса:

1. Проверяет наличие символа '@'.
2. Убирает пробелы вокруг '@'.
3. Валидирует формат: user@domain.tld.
4. Разрешает только латинские буквы, цифры, точки, дефисы и подчеркивания

Регулярное выражение: "^[a- zA- Z0- 9._%+-]+@[a- zA- Z0- 9.-]+\\.[a- zA- Z]{2,}$"

Строка должна начинаться с одного или более символов из набора [латинские буквы, цифры, точка, подчеркивание, процент, плюс, дефис], затем символ '@', затем один или более символов из набора [латинские буквы, цифры, точка, дефис], затем точка, затем минимум 2 латинские буквы, и затем конец строки

2.2.6.5. bool ContactValidator::isValidPhoneFormat(const QString &phone)

Простая проверка формата телефона без детализации ошибок. Обертка над validatePhoneWithDetails(), возвращающая только булев результат.

2.2.6.6. validatePhoneWithDetails

Детальная проверка формата телефона с возвращением сообщения об ошибке. Выполняет комплексную валидацию:

1. Проверка парности скобок.
2. Отсутствие двойных дефисов/пробелов.
3. Соответствие основному формату номера.
4. Проверка длины (7- 15 цифр).
5. Специальные требования для российских номеров (11 цифр).
6. Проверка допустимых символов.
7. Корректность позиции знака '+'.

Регулярное выражение "(^(\+?[1- 9]\d{0,2}[\s\-]*)?(\(?\d{2,5}\)?[\s\-]*)?\d{2,}[\s\-]*\d{2,}[\s\-]*\d{0,}$)");

2.2.6.7. isValidBirthDate

Проверяет корректность даты рождения. Дата должна быть:

1. Валидной (корректный день для месяца).
2. В прошлом (не сегодня и не в будущем).

2.2.7. Описание функций класса MainWindow

2.2.7.1. MainWindow(QWidget *parent)

Конструктор главного окна приложения. Выполняет инициализацию интерфейса и данных при запуске программы.

Последовательность действий:

1) Вызов конструктора родительского класса QMainWindow.
2) Создание модели данных для таблицы (QStandardItemModel).
3) Вызов метода setupUI() для настройки интерфейса. Внутри этого метода создаются следующие связи:

   - connect(searchBox, SIGNAL(textChanged(const QString &)), this, SLOT(onSearchTextChanged(const QString &))); - изменение текста в поле поиска вызывает фильтрацию.
   - connect(searchFieldComboBox, SIGNAL(currentIndexChanged(int)), this, SLOT(onSearchFieldChanged(int))); - изменение выбранного поля поиска вызывает обновление фильтрации.

4) Вызов метода createActions() для создания действий. Внутри этого метода создаются следующие связи:

- connect(actAdd, SIGNAL(triggered()), this, SLOT(onAddContact()));
- connect(actEdit, SIGNAL(triggered()), this, SLOT(onEditContact()));
- connect(actDelete, SIGNAL(triggered()), this, SLOT(onDeleteContact()));
- connect(actSave, SIGNAL(triggered()), this, SLOT(onSave()));
- connect(actLoad, SIGNAL(triggered()), this, SLOT(onLoad()));

5) Вызов метода createMenus() для создания меню.
6) Вызов метода updateTable() для первичного заполнения таблицы данными.

2.2.7.2. ~MainWindow()

Деструктор главного окна. Использует синтаксис = default для автоматической генерации деструктора компилятором. Освобождает все ресурсы, созданные в конструкторе.

2.2.7.3. setupUI

Создает и настраивает пользовательский интерфейс приложения. Функция выполняет:

1. Создание центрального виджета и главной компоновки.
2. Инициализацию элементов поиска (выпадающий список и поле ввода).
3. Настройку таблицы для отображения контактов.
4. Организацию компоновки (layout) элементов.
5. Подключение сигналов и слотов для поиска

2.2.7.4. onSearchFieldChanged

Обработчик изменения выбранного поля для поиска. При смене поля поиска в выпадающем списке автоматически запускает поиск с текущим текстом, чтобы отфильтровать контакты по новому критерию.

Последовательность действий:

1) Получить текущий текст из поля ввода поиска (searchBox- >text()).
2) Вызвать основной слот фильтрации onSearchTextChanged(), передав ему полученный текст.
3) Это приводит к немедленному пересчёту и отображению контактов, соответствующих новому выбранному полю и текущему тексту поиска.

2.2.7.5. onSearchTextChanged

Фильтрация контактов в таблице на основе введенного текста. Реализует гибкую систему поиска:

1. Принимает текст из поля поиска.
2. Определяет выбранное поле поиска.
3. Для телефонов: извлекает только цифры для поиска.
4. Для остальных полей: обычный текстовый поиск.
5. Скрывает строки, не соответствующие критериям поиска.

Логика работы:

- Режим "Все поля": поиск по всем 7 столбцам таблицы.
- Выбранное поле: поиск только по указанному столбцу.
- Обработка телефонов: поиск по цифрам номеров, игнорируя форматирование.
- Регистронезависимость: toLower() для текстовых полей.
- Динамическая фильтрация: скрытие строк в реальном времени.

2.2.7.6. createActions

Создание действий (actions) и добавление их на панель инструментов. Создает 5 основных действий для работы с контактами и файлами, подключает их к соответствующим слотам и размещает на тулбаре.

Последовательность действий:

1) Создать 5 объектов QAction с текстом: "Добавить", "Редактировать", "Удалить", "Сохранить", "Загрузить".
2) Подключить сигнал triggered() каждого действия к соответствующему слоту класса MainWindow.
3) Создать панель инструментов (QToolBar) с названием "Контакты".
4) Добавить на панель инструментов созданные действия в порядке: Добавить, Редактировать, Удалить, разделитель, Загрузить, Сохранить.

2.2.7.7. createMenus

Создание меню приложения с теми же действиями, что и на панели инструментов. Создает главное меню с двумя подменю: "Файл" (операции с файлами) и "Контакт" (операции с контактами).

Последовательность действий:

1) Создать строку меню (QMenuBar) и установить её как главное меню окна.
2) Добавить в строку меню пункт "Файл" (QMenu).
3) В меню "Файл" добавить два действия: "Загрузить" (привязанное к onLoad) и "Сохранить" (привязанное к onSave).
4) Добавить в строку меню пункт "Контакт" (QMenu).
5) В меню "Контакт" добавить три действия: "Добавить" (к onAddContact), "Редактировать" (к onEditContact), "Удалить" (к onDeleteContact).

2.2.7.8. updateTable

Обновление таблицы контактов на основе списка contacts. Очищает таблицу и заполняет её данными из списка контактов, сохраняя исходную дату рождения в пользовательских данных.

Последовательность действий:

1) Удалить все существующие строки из модели данных таблицы (model).
2) Для каждого контакта (Contact) в списке contacts:

    - Вызвать метод контакта toDisplayList() для получения списка строк (QStringList).

- Создать список элементов (QList<QStandardItem*>).

3) Для каждой строки в полученном списке:

- Создать новый элемент таблицы (QStandardItem) с этим текстом.
- Особый случай для столбца "Дата рождения" (пятый элемент): сохранить оригинальный объект QDate (контакта birthDate) в пользовательские данные элемента (Qt::UserRole) для корректной сортировки.
- Добавить элемент в строку.

4) Добавить сформированную строку в модель данных (model).

2.2.7.9. getCurrentRow

Последовательность действий:

1) Получение индекса текущей выбранной строки в таблице. Возвращает индекс строки, выбранной пользователем, или - 1 если ничего не выбрано.
2) Получить у таблицы (tableView) список индексов выбранных строк через selectionModel()- >selectedRows().
3) Проверить, пуст ли полученный список.
4) Если список пуст - вернуть - 1.
5) Если список не пуст - вернуть номер строки (row()) из первого (единственного) индекса в списке.

2.2.7.10. getPhoneType

Отображение диалога выбора типа телефона. Показывает стандартный Qt диалог с выпадающим списком для выбора типа телефона из предопределенного набора.

Последовательность действий:

1) Создать список строк (QStringList) с предопределёнными типами телефонов: {"рабочий", "домашний", "служебный", "мобильный"}.
2) Определить индекс выбранного по умолчанию элемента: Если переданный currentType найден в списке - использовать его индекс. Иначе - использовать индекс 0 (первый элемент).
3) Отобразить стандартный диалог выбора элемента (QInputDialog::getItem), передав: Родительский виджет. Заголовок окна. Текст- подсказка. Список типов. Индекс выбранного по умолчанию. Флаг невозможности редактирования списка. Указатель на флаг успешного завершения (ok).
4) Проверить флаг ok. Если он false или результат пустая строка - вернуть пустую строку.
5) Иначе - вернуть выбранный пользователем тип телефона.

2.2.7.11. editPhones

Диалог управления списком телефонов контакта с возможностью добавления, редактирования и удаления. Создает модальное диалоговое окно для работы со списком телефонов контакта. Использует QListWidget для отображения и лямбда- функции для обработки действий. Принцип работы:

1. Инициализация: Диалог получает ссылку на список телефонов контакта.
2. Отображение: Существующие телефоны показываются в QListWidget.
3. Модальность: Диалог блокирует основное окно до закрытия.
4. Работа с данными: Все изменения применяются напрямую к переданному списку phones.
5. Валидация: Номера телефонов проверяются через ContactValidator.
6. Нормализация: Номера сохраняются в каноническом формате (только цифры).

Последовательность действий (создание связей в диалоге):

1) Создание диалогового окна (QDialog).
2) Создание QListWidget и заполнение его текущими телефонами.
3) Создание кнопок "Добавить", "Редактировать", "Удалить" и QDialogButtonBox (OK, Cancel).
4) Создание связей для кнопок управления:

   connect(btnAdd, &QPushButton::clicked, [&]() { ... }); - нажатие открывает диалог ввода нового номера.

   connect(btnEdit, &QPushButton::clicked, [&]() { ... }); - нажатие открывает диалог редактирования выбранного номера.

   connect(btnRemove, &QPushButton::clicked, [&]() { ... }); - нажатие удаляет выбранный номер из списка.

5) Создание стандартных связей для кнопок OK/Cancel:

   connect(buttonBox, SIGNAL(accepted()), &dialog, SLOT(accept()));

   connect(buttonBox, SIGNAL(rejected()), &dialog, SLOT(reject()));

6) Отображение диалога и возврат результата.

2.2.7.12. editContact

Универсальный диалог для добавления или редактирования контакта с полной валидацией данных.Создает форму с 7 полями ввода, выполняет комплексную валидацию всех данных, управляет телефонами через отдельный диалог и обрабатывает результаты.
Ключевые особенности:

1. Универсальность: Один диалог для добавления (isNew=true) и редактирования (isNew=false).
2. Пошаговая валидация: Каждое поле проверяется последовательно с фокусом на первой ошибке.
3. Пользовательский опыт: Подробные сообщения об ошибках с примерами правильного формата.
4. Временные данные: Телефоны хранятся в tempPhones до успешной валидации всех полей.
5. Отмена изменений: При нажатии Cancel все изменения отбрасываются.

Последовательность действий (создание связей в диалоге):

1) Создание диалогового окна (QDialog).
2) Создание и настройка всех виджетов ввода (поля QLineEdit, QDateEdit, кнопка управления телефонами).
3) Создание связей для кнопки управления телефонами:

   connect(phonesButton, &QPushButton::clicked, [&]() { editPhones(tempPhones); }); - нажатие открывает диалог управления телефонами.

4) Создание кнопок QDialogButtonBox (OK, Cancel).
5) Создание кастомной связи для кнопки OK:

   connect(buttonBox, &QDialogButtonBox::accepted, [&]() { ... }); - внутри лямбда-функции выполняется полная валидация всех полей, и только при успехе вызывается dialog.accept().

6) Создание связи для кнопки Cancel:

   connect(buttonBox, &QDialogButtonBox::rejected, [&]() { dialog.reject(); });

7) Отображение диалога и возврат результата.

2.2.7.13. onAddContact

Обработчик добавления нового контакта. Создает пустой контакт, вызывает диалог редактирования и, если пользователь подтвердил ввод, добавляет контакт в список и обновляет таблицу. Блок- схема представлена на рисунке 1.

Последовательность действий:

1) Создать новый пустой объект Contact.
2) Вызвать диалог редактирования контакта (editContact) для этого объекта, установив флаг isNew = true.
3) Если диалог завершился подтверждением (пользователь нажал OK и все данные прошли валидацию): Добавить созданный контакт в конец списка contacts. Вызвать updateTable() для отображения нового контакта в таблице.

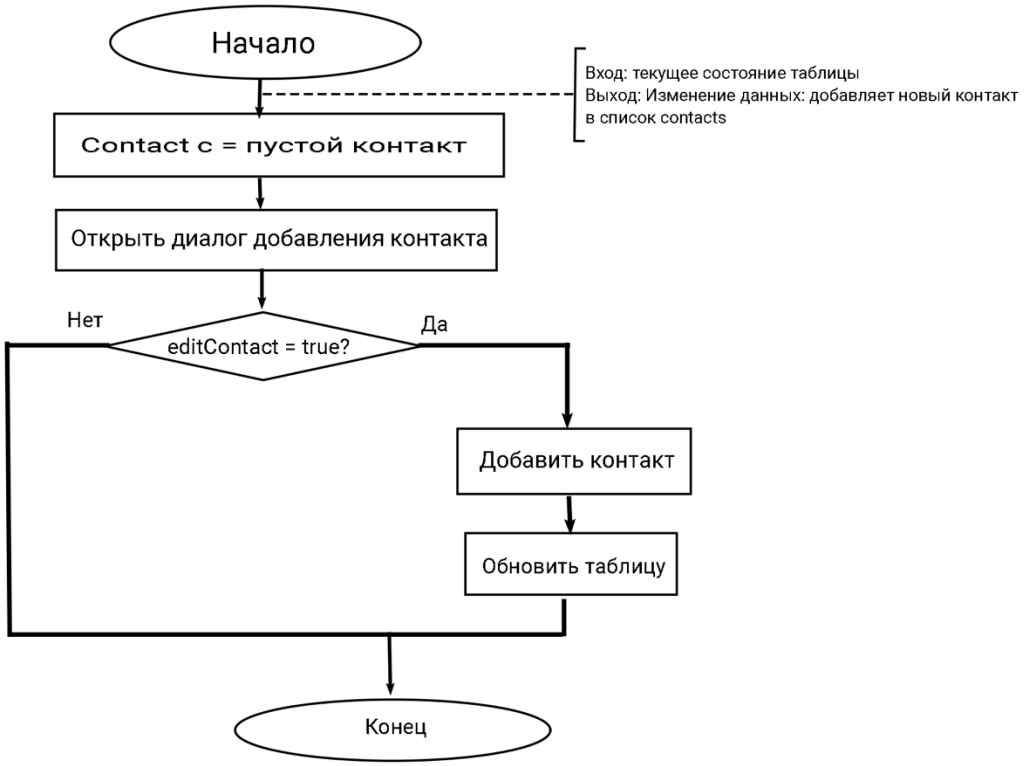

Рис.1. Блок- схема onAddContact().

2.2.7.14. onEditContact

Обработчик редактирования существующего контакта. Проверяет выбор контакта в таблице, открывает диалог редактирования с данными выбранного контакта и обновляет таблицу при успешном изменении. Блок- схема представлена на рисунке 2.

Последовательность действий:

1) Получить индекс выбранной строки в таблице, вызвав getCurrentRow().
2) Если возвращён - 1 (ничего не выбрано): Показать пользователю предупреждающее сообщение с просьбой выбрать контакт. Завершить выполнение функции.
3) Вызвать диалог редактирования контакта (editContact) для контакта, находящегося в списке contacts под полученным индексом, установив флаг isNew = false.
4) Если диалог завершился подтверждением (данные были изменены и прошли валидацию): Вызвать updateTable() для обновления отображения изменённого контакта в таблице.

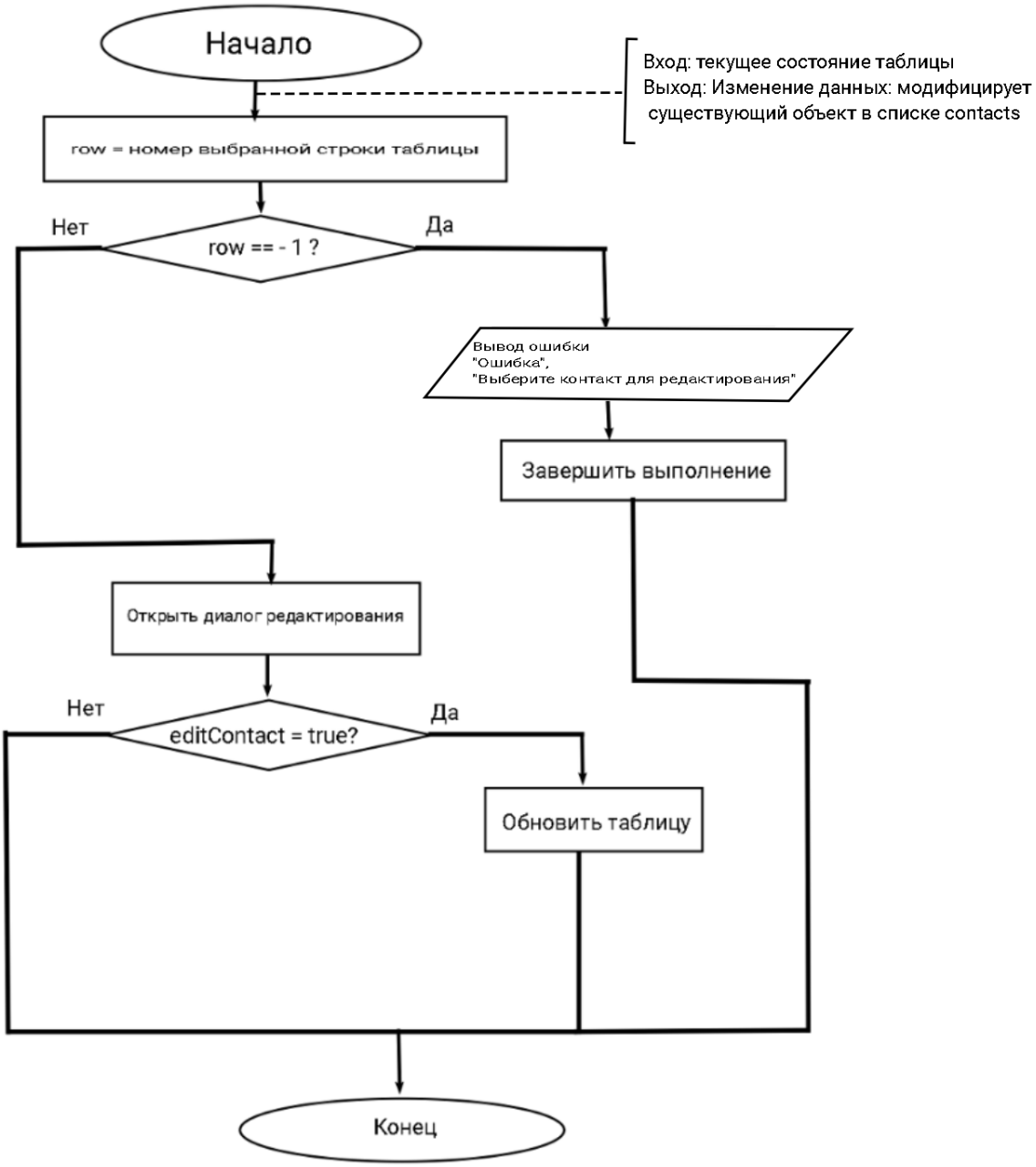

Рис.2. Блок- схема onEditContact

2.2.7.15. onDeleteContact

Обработчик удаления контакта с подтверждением. Проверяет выбор контакта, запрашивает подтверждение удаления и удаляет контакт из списка при положительном ответе. Блок- схема представлена на рисунке 3.

Последовательность действий:

1) Получить индекс выбранной строки в таблице, вызвав getCurrentRow().
2) Если возвращён - 1 (ничего не выбрано): Показать пользователю предупреждающее сообщение с просьбой выбрать контакт. Завершить выполнение функции.

3) Показать диалог подтверждения (QMessageBox::question) с вопросом, действительно ли пользователь хочет удалить контакт, и вариантами ответа "Да"/"Нет".
4) Если пользователь выбрал "Да": Удалить контакт под полученным индексом из списка contacts. Вызвать updateTable() для обновления таблицы (удалённый контакт исчезнет).

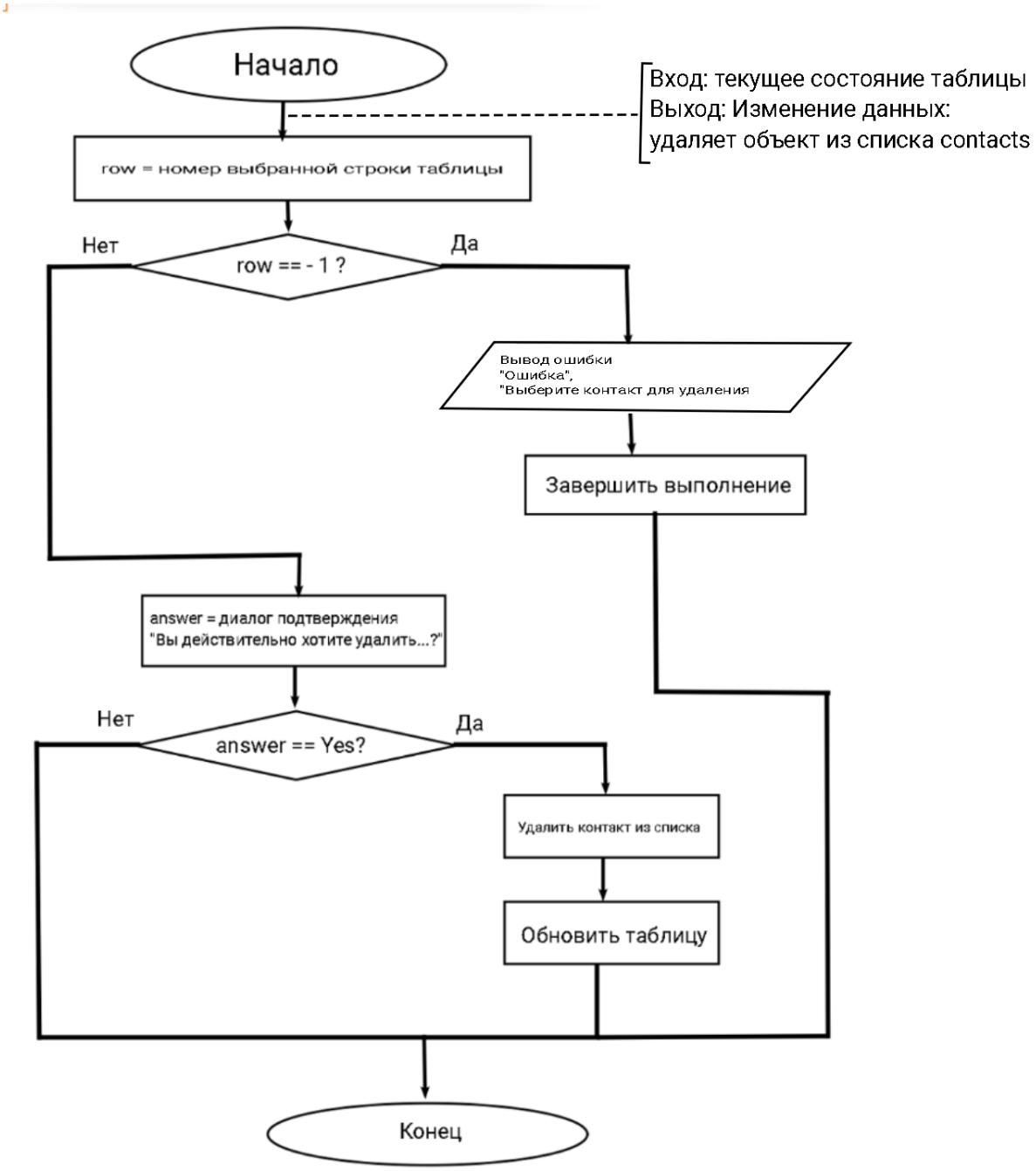

Рис.3. Блок- схема onDeleteContact().

2.2.7.16. void onSave

Обработчик сохранения справочника в файл. Открывает диалог выбора файла для сохранения и вызывает функцию сохранения данных в JSON формате.

Последовательность действий:

1) Открыть стандартный диалог сохранения файла (QFileDialog::getSaveFileName), предложив пользователю выбрать имя и расположение JSON- файла.
2) Если пользователь выбрал имя файла (вернувшаяся строка не пустая): Вызвать функцию saveToFile(), передав ей выбранное имя файла для выполнения операции сохранения.

2.2.7.17. onLoad

Обработчик загрузки справочника из файла. Открывает диалог выбора файла для загрузки, вызывает функцию чтения данных из JSON файла и обновляет таблицу.

Последовательность действий:

1) Открыть стандартный диалог открытия файла (QFileDialog::getOpenFileName), предложив пользователю выбрать JSON- файл для загрузки.
2) Если пользователь выбрал файл (вернувшаяся строка не пустая): Вызвать функцию loadFromFile(), передав ей путь к выбранному файлу. Эта функция загрузит данные и заменит ими текущий список contacts. Вызвать updateTable() для отображения загруженных контактов в таблице.

2.2.7.18. saveToFile

Сохранение списка контактов в JSON файл. Сериализует все контакты в JSON формат с сохранением всех полей и телефонов, записывает в файл с отступами для читаемости.

Последовательность действий:

1) Создаётся пустой JSON- массив (QJsonArray).
2) Для каждого контакта в списке contacts выполняется:

    - Создаётся JSON- объект для контакта (QJsonObject).
    - В этот объект записываются все поля контакта как пары "ключ- значение": lastName, firstName, patronymic, address, email - как строки (QString). birthDate - как строка в стандартном ISO- формате (Qt::ISODate), полученная методом QDate::toString().
    - Для хранения телефонов создаётся вложенный JSON- массив (QJsonArray).
    - Для каждого телефона (PhoneInfo) в списке контакта: Создаётся JSON- объект для телефона. В него записываются type и number как строки. Объект телефона добавляется во вложенный массив.
    - Вложенный массив телефонов добавляется в объект контакта по ключу "phones".
    - Готовый объект контакта добавляется в корневой JSON- массив.

3) Создаётся JSON- документ (QJsonDocument) из сформированного массива.
4) Создаётся объект QFile для записи по указанному пути.
5) Файл открывается для записи (QIODevice::WriteOnly).
6) Если файл успешно открыт:

- JSON- документ преобразуется в отформатированный (с отступами, QJsonDocument::Indented) текст и записывается в файл.
- Файл закрывается.
- Пользователю выводится сообщение об успешном сохранении.

7)Если открыть файл не удалось, пользователю выводится сообщение об ошибке.

2.2.7.19. loadFromFile

Загрузка списка контактов из JSON файла. Читает JSON файл, парсит данные, создает объекты контактов с телефонами и загружает их в список.

Последовательность действий:

1) Создаётся объект QFile для работы с файлом по переданному пути.
2) Попытка открыть файл только для чтения (QIODevice::ReadOnly). Если открыть не удалось - выводится сообщение об ошибке и функция завершается.
3) Все данные из файла считываются в массив байт и передаются в статический метод QJsonDocument::fromJson() для создания JSON- документа.
4) Файл закрывается.
5) Проверяется, является ли корневой элемент документа массивом (isArray()). Если нет - выводится сообщение об ошибке некорректного формата и функция завершается.
6) Текущий список контактов в приложении полностью очищается (contacts.clear()).
7) Для каждого элемента (QJsonValue) в JSON- массиве выполняется:

    - Значение преобразуется в JSON- объект (QJsonObject).
    - Создаётся новый пустой объект Contact.
    - Заполняются текстовые поля контакта (lastName, firstName, patronymic, address, email) соответствующими строками из JSON- объекта.
    - Поле birthDate восстанавливается из строки в формате ISO (Qt::ISODate) с помощью QDate::fromString().
    - Для каждого элемента вложенного JSON- массива "phones": создаётся новый объект PhoneInfo, заполняются его поля type и number из соответствующего JSON- объекта, телефон добавляется в список phones текущего контакта, полностью собранный контакт добавляется в общий список contacts.

8) После завершения цикла выводится информационное сообщение с количеством успешно загруженных контактов.

2.3. Задача 3. Добавление поддержки PostgreSQL

В рамках третьей задачи требовалось расширить функциональность приложения, добавив возможность хранения данных в базе данных PostgreSQL, при этом сохранив файловое хранилище как альтернативный вариант.

2.3.1. Архитектура хранилищ

Для поддержки двух типов хранилищ была разработана абстрактная архитектура на основе интерфейса IRepository.

1) Абстрактный класс IRepository определяет единый контракт для всех типов хранилищ.
2) Класс FileRepository является рефакторингом существующего файлового хранилища из Задачи 1. Он реализует интерфейс IRepository и обеспечивает:

    - Сохранение контактов в текстовый файл с разделителями
    - Загрузку контактов из файла с валидацией данных
    - Кэширование загруженных данных для повышения производительности

Основные методы: saveToFile() / loadFromFile() - статические методы для работы с файлами. Кэш cache хранит загруженные контакты для быстрого доступа

3) Класс PostgresRepository реализует работу с базой данных PostgreSQL через библиотеку libpq. Основные методы класса:

    - PostgresRepository(connStr). Конструктор, устанавливает соединение с БД и создаёт таблицы при их отсутствии
    - createTablesIfNotExist(). Создание таблиц, если они не существуют
    - addContact(). Добавление контакта и связанных с ним телефонов в БД
    - updateContact(). Обновление данных контакта с перезаписью списка телефонов
    - deleteContact(). Удаление контакта (телефоны удаляются автоматически благодаря CASCADE)
    - loadAll(). Загрузка всех контактов с их телефонами
    - escapeString(). Экранирование строк для безопасного выполнения SQL-запросов

    Особое внимание потребовала работа с датами. В программе дата хранится в формате DD.MM.YYYY, а PostgreSQL ожидает формат YYYY- MM- DD. Для этого были реализованы вспомогательные методы formatDateForPostgres и formatDateFromPostgres.

4) Класс RepositoryFactory реализует паттерн «Фабрика» для создания экземпляров хранилищ нужного типа. При запуске приложения пользователю предлагается выбрать тип хранилища.

2.3.2. Интеграция в основное приложение

В main.cpp были внесены следующие изменения:

1) Выбор хранилища при запуске - пользователь выбирает тип хранилища, после чего создаётся соответствующий репозиторий.
2) Загрузка данных - при старте программы данные автоматически загружаются из выбранного хранилища.
3) Сохранение при выходе - при завершении работы программа предлагает сохранить изменения.

4) Новый пункт меню - добавлен пункт 9 «Сменить тип хранилища», позволяющий переключаться между файловым и PostgreSQL- хранилищем во время работы программы.

2.3.3. Обработка ошибок подключения

При выборе PostgreSQL-хранилища программа проверяет возможность подключения к базе данных. Если подключение не удалось (например, сервер не запущен или база данных отсутствует), программа выводит сообщение об ошибке и автоматически переключается на файловое хранилище, гарантируя работоспособность приложения.

2.3.4. Сборка проекта

Для сборки программы с поддержкой PostgreSQL были выполнены следующие шаги:

1) Установка PostgreSQL и библиотеки libpq с помощью brew
2) Указание пути в PhoneBook.pro
3) Сборка проекта с помощью qmake

3. Тестирование приложения

Покажем работоспособность программы в QT на примерах. При запуске приложения, пользователю доступно стартовое окно с заголовком “Телефонный справочник”. Ниже панель меню с кнопками “Добавить”,” Редактировать”,” Удалить”,” Загрузить”,” Сохранить”. Пока активны только кнопки “Добавить” и “Загрузить”, так как на данном этапе еще нет ни одного контакта. Далее расположены активные поля ввода и выбора критерия для поиска (Рис.4).

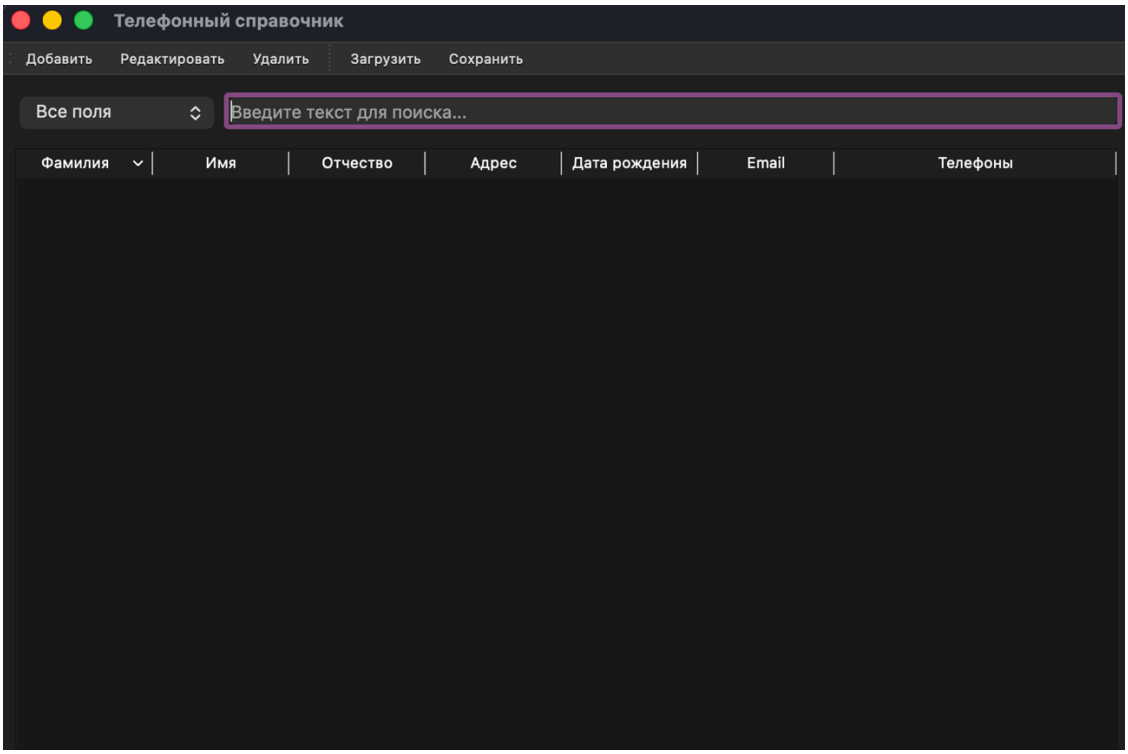

Рис.4. Стартовое окно.

Далее рассмотрим все возможности работы программы.

1. Добавление контакта.

Для добавления нового контакта, необходимо нажать кнопку “Добавить” (Рис.5). Далее появляется окно для заполнения данных о новом контакте (Рис.6). При добавлении номера телефона появляется отдельное окно, после ввода номера можно выбрать тип телефона (Рис.7). При добавлении нового контакта, все поля проверяются на корректность ввода. В случае ошибок появляется предупреждение об ошибке с информацией что именно не правильно (Рис.8).

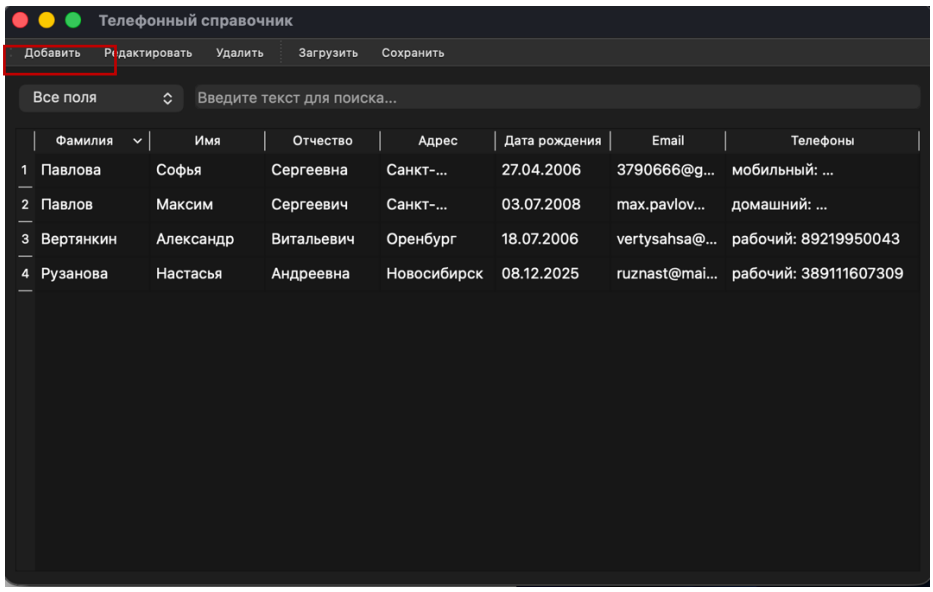

Рис.5. Главное меню программы, кнопка “Добавить”.

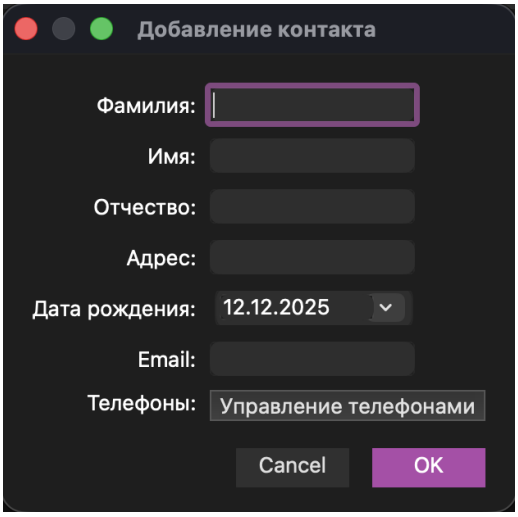

Рис.6. Окно для добавления нового контакта.

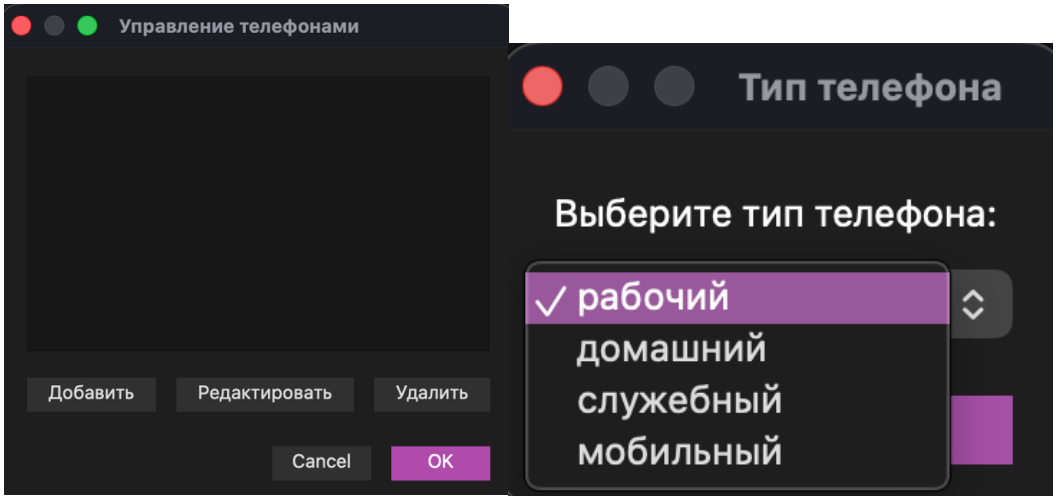

Рис.7. Окно для добавления номера телефона.

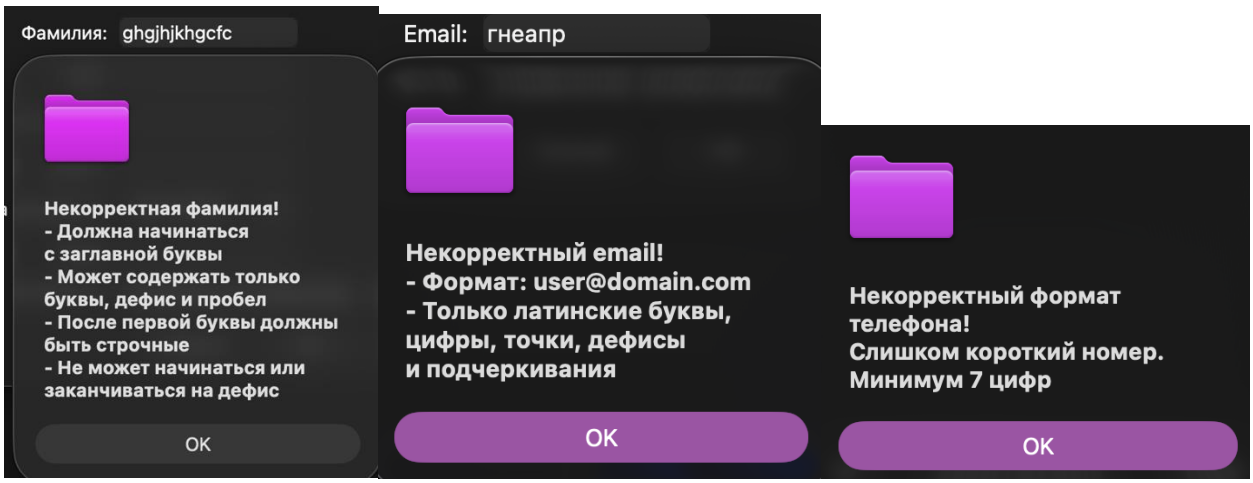

Рис.8. Обработка некорректного ввода.

2. Удаление контакта.

Для удаления контакта, необходимо выбрать контакт нажатием и с помощью кнопки “Удалить” внести необходимые изменения, при этом необходимо будет подтвердить свои действия (Рис.9).

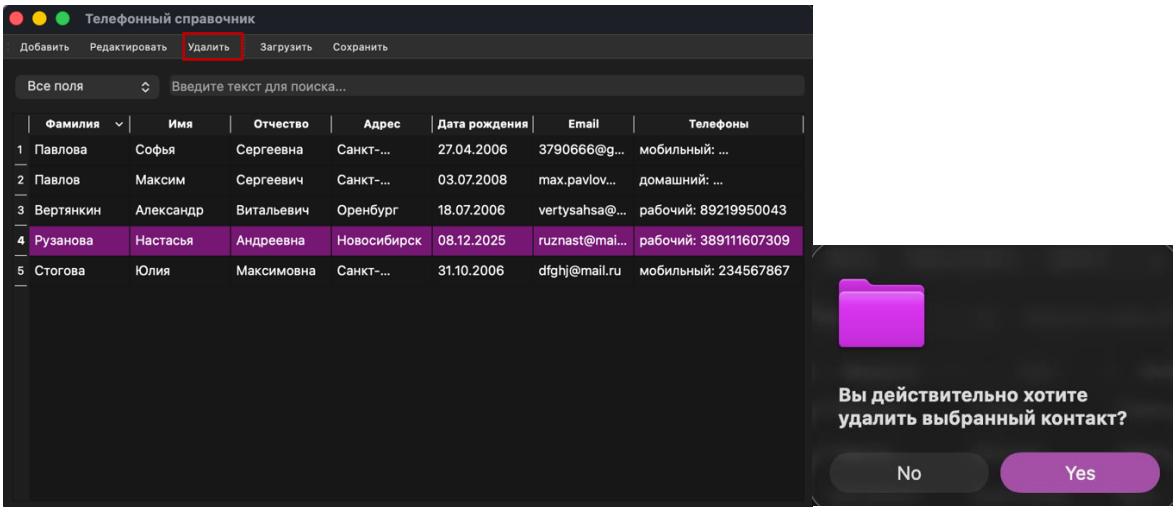

Рис.9. Удаление контакта.

3. Редактирование контакта.

Для редактирования контакта, необходима выбрать контакт нажатием и с помощью кнопки “Редактировать” внести необходимые изменения (Рис.10). При ошибках в редактировании и добавлении контакта, появляются предупреждения с информацией о том, что именно было введено неправильно (Рис.11).

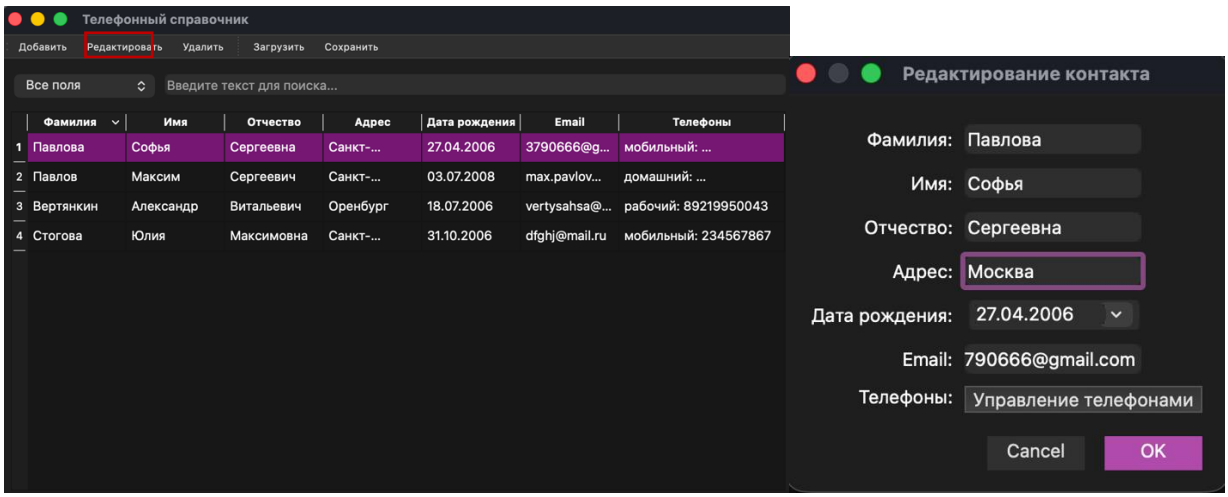

Рис.10. Редактирование контакта.

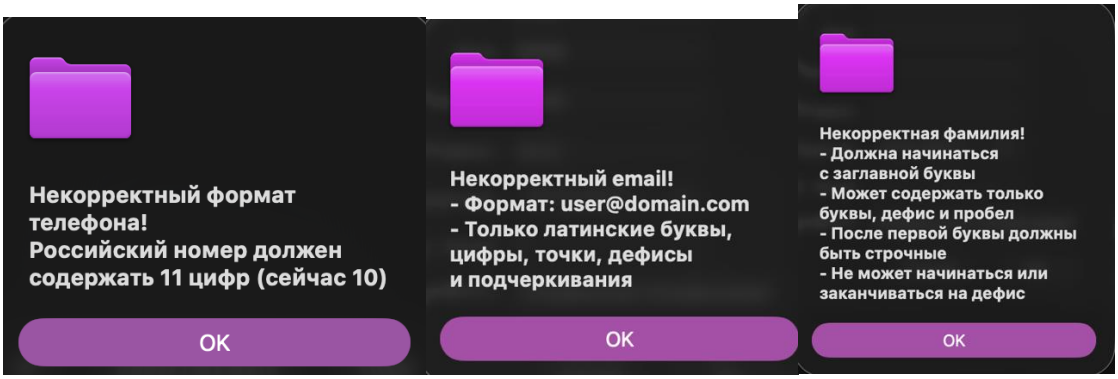

Рис.11. Информация о некорректном вводе.

4. Поиск контактов.

Поиск контактов можно производить по любым полям (Рис.12). При вводе каждого нового символа находятся все контакты, в полях которых есть данное сочетание символов, остальные контакты скрываются (Рис.13).

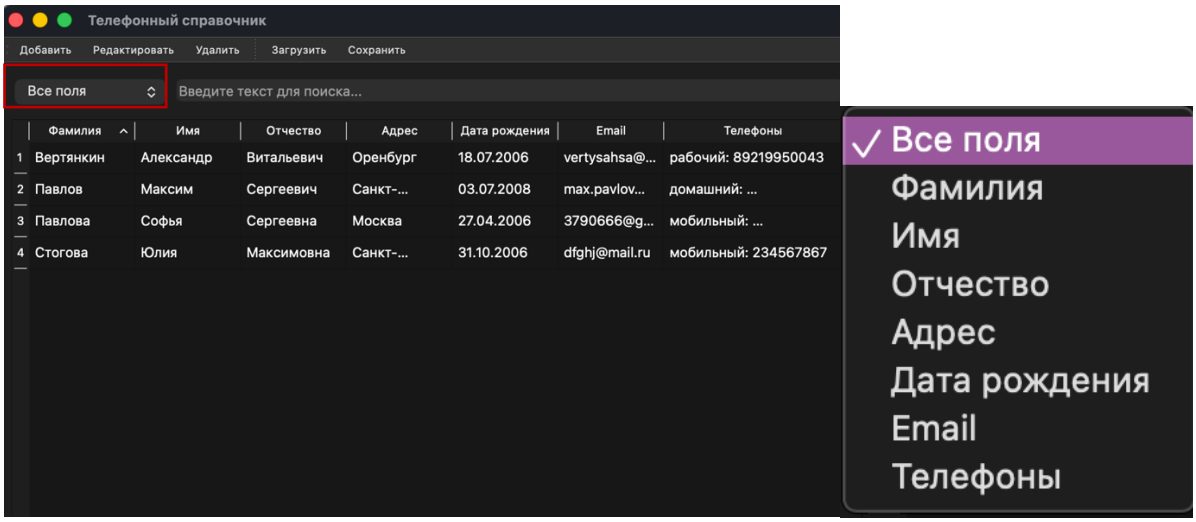

Рис.12. Поиск по любым полям.

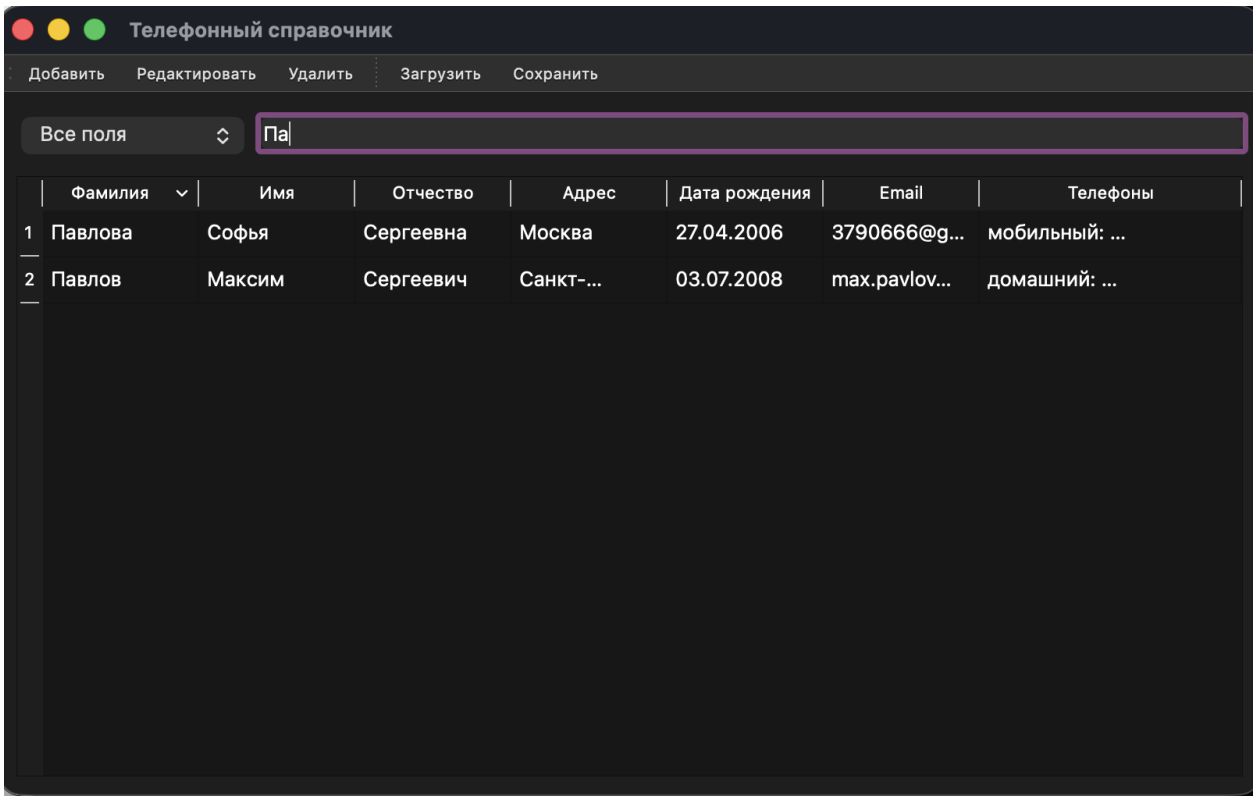

Рис.13. Поиск контакта.

5. Сохранение списка контактов.

Для сохранения списка контактов, необходимо нажать кнопку “Сохранить” и ввести название .json файла для сохранения (Рис.14).

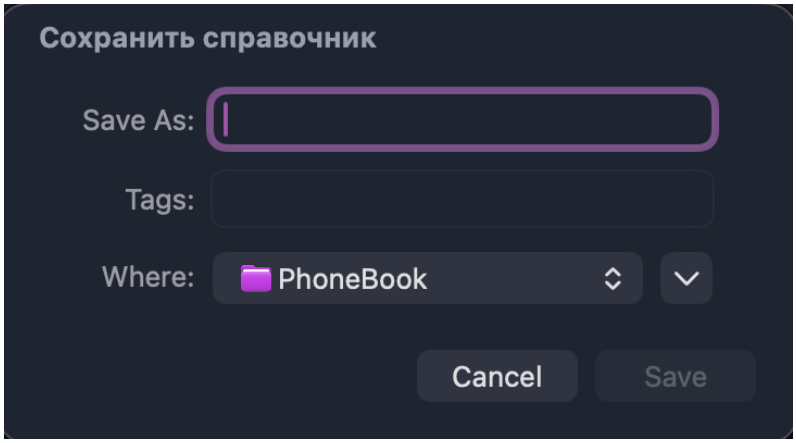

Рис.14. Сохранение справочника.

6. Загрузка списка контактов.

При загрузке списка контактов, все старые удаляются, а контакты из файла появляются вместо них. При этом сообщается сколько контактов было добавлено (Рис.15).

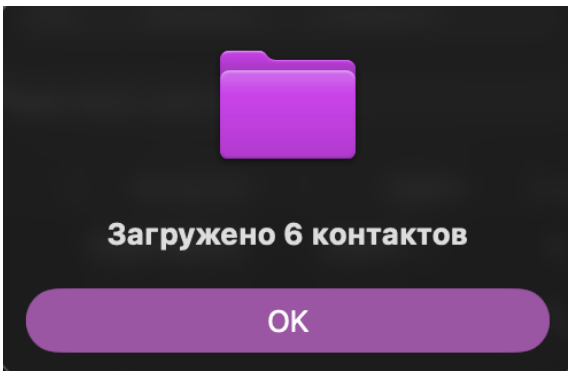

Рис.15. Загрузка справочника.

Заключение

В ходе выполнения работы было успешно разработано приложение «Телефонный справочник» с использованием фреймворка Qt. Все поставленные задачи выполнены в полном объёме, достигнуты следующие результаты:

1. Разработана структура данных для хранения контактов:

2. Реализован модуль валидации и нормализации данных:

3. Создан графический пользовательский интерфейс:

4. Реализованы диалоговые окна для операций с контактами:

5. Разработана система динамического поиска и фильтрации:

6. Реализована система сериализации и работы с файлами:

7. Обеспечена корректность данных и логики приложения:

Ожидаемый функционал программы полностью реализован. Приложение предоставляет удобный интерфейс для управления контактами, поддерживает валидацию данных, динамический поиск, сортировку и работу с файлами.

Технические характеристики:

- Язык программирования: C++
- Фреймворк: Qt 6.9.2
- Компилятор: Apple Clang 17.0.0 (clang- 1700.4.4.1)
- Операционная система: macOS 15.x (Sequoia)
- Архитектура: Apple Silicon (arm64)
- Инструмент сборки: QMake 3.1

Список использованных источников:

1. Официальная документация Qt 6.9 (Электронный ресурс). URL: https://doc.qt.io/qt- 6/ (дата обращения: 20.11.2025).

2. Документация классов Qt Widgets (Электронный ресурс). URL: https://doc.qt.io/qt-6/qtwidgets- index.html (дата обращения: 20.11.2025).

3. Документация по классу QRegularExpression (Электронный ресурс). URL: https://doc.qt.io/qt- 6/qregularexpression.html (дата обращения: 20.11.2025).

4. Документация по работе с JSON в Qt (Электронный ресурс). URL: https://doc.qt.io/qt-6/json.html (дата обращения: 20.11.2025).